Coursework 1: Sentiment Analysis from Tweets

Carlota Cortés Mir ; Student ID: 250711969

Question 1 (10 marks): Simple Feature Extraction

To complete this task, I implemented the `to_feature_vector()` function, which takes a preprocessed list of tokens and returns a dictionary of features and weights. The features are the unique tokens that appear in the text, and the weights represent the number of times each word appears in the input text. This approach follows a simple bag-of-words representation.

The function iterates through each token and increments its count in the `feature_vector` dictionary. Additionally, it updates the `global_feature_dict` variable to keep track of all tokens in the dataset, although this is not required for this task.

Question 2 (20 marks): Cross-Validation on Training Data

To evaluate the classifier's performance, I completed the implementation of the `cross_validate()` function, which performs 10-fold cross-validation on the training data. This approach provides a more reliable estimate than a single train/test split by averaging results across ten independent folds.

The dataset is shuffled and split into ten equally sized folds. For each iteration, the classifier trains on nine folds and validates on one. Performance metrics (precision, recall, F1-score, and accuracy) are computed using sklearn.metrics, comparing the true and predicted labels. In the end, the results are averaged across folds to produce the final `cv_results` dictionary. Overall, the classifier performed well, achieving an average F1-score of 0.83.

Question 3 (20 marks): Error Analysis

The goal of this task was to identify where the classifier struggles and uncover errors. For this analysis, there isn't a specific formula behind, it's based on inspecting misclassifications to find patterns or common characteristics.

I implemented an `error_analysis()` function that reuses the first fold of the cross-validation setup to create a simple 90/10 train–test split. The function trains the classifier, predicts labels, plots the confusion matrix using the provided `confusion_matrix_heatmap()` function, and writes two text files containing 15 examples of false positives and false negatives with their raw text. I modified the `split_and_preprocess()` function to retain raw text for each sample.

Confusion Matrix Results: TP = 1,532; TN = 671; FP = 289; FN = 191. These results suggest a slight bias toward the positive class, as it tends to predict more positives than negatives.

Examples of False Negatives:

1. "Napoleon dynamite is on and im not going to school tomorrow. #fantastic"
2. "not a bit ashamed to say I have preplanned LOTS of jokes about the Queen/parasitology to troll my Tory royalist classmates tomorrow"

Examples of False Positives:

1. "Clearly you haven't upgraded to Windows 10 yet. Good night & good luck! #Windows10Fail"
2. "Oh how I love the irony of Muslims, it's a concept they just can't quite grasp!"

These examples show that the classifier doesn't work well with context understanding, it performs more of a word-level sentiment detection. It struggles particularly with:

- Handling negations and double negatives
- Sarcasm and humor
- Implicit or subtle sentiment without strong sentiment words

Question 4 (50 marks): Improving Model Performance

This question focused on improving the classifier's performance by optimising pre-processing and feature extraction. Each modification was tested systematically using 10-fold cross-validation, isolating the effect of each change and primarily focusing on the F1-score metric.

I designed the implementation to allow flexible experimentation with configurable Boolean variables, avoiding code duplication and enabling quick experiment configuration.

Approaches Tested:

1. **Lowercasing:** Converts tokens to lowercase, reducing redundancy and improving generalisation. In sentiment analysis, we don't have to deal with the problem of "U.S" to "us" because "Love" has the same sentiment value as "love".
2. **Stopword removal:** Removes common non-informative words as they do not hold any sentiment value and mostly add feature space and noise. Therefore, we can get rid of them before feature extraction using NLTK, which provides a list of stop words.
3. **Negation handling:** In the error analysis, the classifier misclassified a lot of negated phrases. So by adding a "_NEG" suffix to tokens following negations it can improve the model's performance.
4. **TF-IDF weighting:** Replaces count-based weights with TF-IDF, which downweights common but uninformative words. As TF-IDF considers both the frequency of a word in a document and its importance across the entire dataset.
5. **N-grams:** In sentiment analysis with unigrams, the classifier normally might predict positive for having the word "like", when it is "don't like". So, if we include bigrams, we'll have features including "don't like" and might improve semantic understanding and classification better.
6. **SVM hyperparameter tuning:** Tune the regularisation parameter C, testing with different values (0.01, 0.1, 1, 10, 20, 30, 40), to help balance variance. Also test class weighting which helps if the data is imbalanced.

Experiments and Observations:

Config	F1-score	Observation
Baseline	0.8314	–
Lowercasing	0.8433	Improved, reduced redundant features
Stopword removal	0.8226	Worse, it may have removed sentiment-bearing words
Lowercase + Stopword removal	0.8307	Slight recovery, but still below baseline
Negation handling	0.8402	Better than baseline
Lowercase + Negation	0.8406	Minor improvement
TF-IDF weighting	0.8637	Major improvement
Lowercase + TF-IDF	0.8653	Further improved
Add bigrams (n=1,2)	0.8657	Major improvement
Lowercase + TF-IDF + N-grams	0.8671	Best combination so far
+ SVM tuning (C=30)	0.8720	Best overall
+ Class weights balanced	0.8705	No further gain

Final best configuration:

- Pre-processing: Lowercasing
- Feature extraction: TF-IDF weighting with unigrams and bigrams
- Classifier: Linear SVM with C=30

Final results on test set: Precision = 0.8665; Recall = 0.8678; F1-score = 0.8662

Conclusions: TF-IDF weighting, n-gram features and SVM hyperparameter tuning contributed the most to the model's improvement. Stopword removal reduced performance, while negation handling provided only minor benefits. Overall, with the best configuration, the model improved its F1-score from 0.83 to 0.87, showing consistent and effective improvement.